

Technical Documentation: TalentMatch AI

Project: AI-Powered Resume Screening & Job Fit Analyzer

1. Project Overview

TalentMatch AI is a serverless application designed to automate HR screening workflows. The system processes uploaded candidate resumes (PDFs), extracts text content, and utilizes a Large Language Model to evaluate the candidate's skills against a provided job description. It generates an objective match percentage, extracts contact details, and flags potential keyword-stuffing via an authenticity score.

2. Technology Stack

- **Backend:** Python 3.10, Flask 3.0.0
- **AI Integration:** Google Gemini 3.5 Flash API (via direct HTTP/REST)
- **Document Parsing:** PyPDF2
- **Frontend:** HTML5, Tailwind CSS, Vanilla JS (Fetch API)
- **Deployment & CI/CD:** Vercel Serverless Functions, GitHub Actions

3. Core Implementations & Logic

- **Secure Credential Management:** API keys and deployment tokens are securely managed using `.env` for local development and GitHub Secrets/Vercel Environment Variables in production.
- **File Parsing Pipeline:** Built a robust text extraction module using `PyPDF2` to handle `multipart/form-data` uploads directly in memory without saving files to the disk.
- **Prompt Engineering & JSON Enforcement:** Configured the LLM prompt to strictly return valid JSON containing specific keys (`match_percentage`, `authenticity_score`, `key_matching_skills`, etc.). Implemented backend parsing logic to clean Markdown delimiters from the AI response before sending it to the client.
- **Automated CI/CD:** Engineered a GitHub Actions workflow (`deploy.yml`) that leverages the Vercel CLI. The pipeline resolves Python versioning and dependency manager (`uv`) requirements on the Ubuntu runner before pushing the build artifacts to production.

4. API Specifications

POST /api/analyze

Content-Type: multipart/form-data

Payload:

- `job_description` (text/plain): The target job requirements.
- `resume` (application/pdf): The candidate's resume file.

Response (200 OK): JSON object containing parsed metrics, skill lists, and recommendations.

Error Handling (400/500): Returns structured JSON error messages for missing fields, PDF parsing failures, or upstream Google API timeouts.

5. Deployment Pipeline

The repository is integrated with GitHub Actions. Upon a push to the `main` branch, the pipeline executes the following steps:

1. Checks out the source code.
2. Sets up Python 3.10 and installs the `uv` package manager.
3. Installs the Vercel CLI globally via npm.
4. Authenticates with Vercel using injected `VERCEL_ORG_ID`, `VERCEL_PROJECT_ID`, and `VERCEL_TOKEN`.
5. Builds and deploys the serverless functions to the production environment.